

genai_conversation.pdf

Today

Write a Java program using OOP that manages a list of tasks. Create a Task class with fields for name, priority, and completed status. Use an ArrayList to store Task objects. Include methods to add tasks, list tasks, and mark tasks as done

A small task-manager in Java is a perfect way to practice clean OOP design. Below is a complete, ready-to-run example with a **Task** class, an **ArrayList<Task>**, and methods to add, list, and mark tasks as done.

Concise takeaway

You'll create a **Task** class with fields for name, priority, and completion status, then manage a list of these objects using an **ArrayList** inside a **TaskManager** class, with methods to add, list, and complete tasks.

Java OOP Task Manager Program

```
java
import java.util.ArrayList;
import java.util.Scanner;

// Represents a single task
class Task {
    private String name;
    private int priority; // 1 = High, 2 = Medium, 3 = Low
    private boolean completed;

    public Task(String name, int priority) {
        this.name = name;
        this.priority = priority;
        this.completed = false;
    }

    public void markCompleted() {
        this.completed = true;
    }

    public boolean isCompleted() {
        return completed;
    }
}
```

```

    public String toString() {
        return "[Name: " + name +
            ", Priority: " + priority +
            ", Completed: " + completed + "]\n";
    }
}

// Manages a list of tasks
class TaskManager {
    private ArrayList<Task> tasks = new ArrayList<>();

    public void addTask(String name, int priority) {
        tasks.add(new Task(name, priority));
    }

    public void listTasks() {
        if (tasks.isEmpty()) {
            System.out.println("No tasks available.");
            return;
        }
        for (int i = 0; i < tasks.size(); i++) {
            System.out.println(i + ". " + tasks.get(i));
        }
    }

    public void markTaskDone(int index) {
        if (index >= 0 && index < tasks.size()) {
            tasks.get(index).markCompleted();
            System.out.println("Task marked as completed.");
        } else {
            System.out.println("Invalid task index.");
        }
    }
}

// Main program with a simple menu
public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        TaskManager manager = new TaskManager();

        while (true) {
            System.out.println("\n1. Add Task");
            System.out.println("2. List Tasks");

```

```

System.out.println("3. Mark Task as Done");
System.out.println("4. Exit");
System.out.print("Choose an option: ");

int choice = scanner.nextInt();
scanner.nextLine(); // consume newline

switch (choice) {
    case 1:
        System.out.print("Task name: ");
        String name = scanner.nextLine();
        System.out.print("Priority (1=High, 2=Medium, 3=Low): ");
        int priority = scanner.nextInt();
        manager.addTask(name, priority);
        break;

    case 2:
        manager.listTasks();
        break;

    case 3:
        System.out.print("Enter task index to mark done: ");
        int index = scanner.nextInt();
        manager.markTaskDone(index);
        break;

    case 4:
        System.out.println("Goodbye.");
        return;

    default:
        System.out.println("Invalid choice.");
}
}
}
}

```

What you might want next

Would you like to extend this program with features such as

- saving tasks to a file,
- sorting tasks by priority, or

- adding due dates?

Pick one and I'll build it out.

Modify TaskManager.java so it includes the correct imports for ArrayList. or Update Main.java so it compiles with Task and TaskManager

You only need **one small fix** depending on which file you want to update. I'll show both options cleanly so your project compiles without errors.

✓ Option 1 — Update TaskManager.java with the correct import

Your `TaskManager` uses an `ArrayList`, so it must import it:

```
java
import java.util.ArrayList;
```

A correct minimal `TaskManager.java` looks like this:

```
java
import java.util.ArrayList;

public class TaskManager {
    private ArrayList<Task> tasks = new ArrayList<>();

    public void addTask(String name, int priority) {
        tasks.add(new Task(name, priority));
    }

    public void listTasks() {
        for (int i = 0; i < tasks.size(); i++) {
            System.out.println(i + ". " + tasks.get(i));
        }
    }

    public void markTaskDone(int index) {
        if (index >= 0 && index < tasks.size()) {
            tasks.get(index).markCompleted();
        }
    }
}
```

✅ Option 2 — Update Main.java so it compiles with Task and TaskManager

If `Task.java` and `TaskManager.java` are in the **same folder** and **no package is declared**, then Main only needs to import **nothing** — it just needs to reference the classes correctly:

```
java
public class Main {
    public static void main(String[] args) {
        TaskManager manager = new TaskManager();
        manager.addTask("Example", 1);
        manager.listTasks();
    }
}
```

If you *did* put them in a package, then Main must import them:

```
java
import mypackage.Task;
import mypackage.TaskManager;
```

🔧 What you may want next

If you want, I can help you reorganize the project into a clean package structure or add features like

- sorting tasks,
- saving tasks to a file, or
- adding a menu-driven UI.

Which direction do you want to take this project?

Add the missing import statements to Main.java so that Scanner works correctly

You only need **one import** for `Scanner` to work correctly in `Main.java`.

✅ Correct import for Scanner

Add this line at the very top of **Main.java**:

```
java
```

```
import java.util.Scanner;
```

Updated Main.java (compiles correctly)

```
java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        TaskManager manager = new TaskManager();
        Scanner scanner = new Scanner(System.in);

        manager.addTask("Example", 1);
        manager.listTasks();
    }
}
```

This ensures the `Scanner` class is recognized by the compiler.

If you want, I can also help you reorganize your files or add features like

- task sorting,
- saving tasks to a file, or
- improving the menu.

Modify `Main.java` so that if the user enters something that is not a number, the program does not crash. Use a `try/catch` around the input

You can make `Main.java` safe by wrapping every numeric input in a **try/catch** so invalid input doesn't crash the program. The key is to catch `InputMismatchException`, clear the bad input, and continue running normally.

Here's a clean, ready-to-compile version of `Main.java` with proper error handling added.

Updated `Main.java` with `try/catch` around numeric input

```
java
import java.util.Scanner;
import java.util.InputMismatchException;

public class Main {
```

```

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    TaskManager manager = new TaskManager();

    while (true) {
        System.out.println("\n1. Add Task");
        System.out.println("2. List Tasks");
        System.out.println("3. Mark Task as Done");
        System.out.println("4. Exit");
        System.out.print("Choose an option: ");

        int choice;

        try {
            choice = scanner.nextInt();
        } catch (InputMismatchException e) {
            System.out.println("Please enter a valid number.");
            scanner.nextLine(); // clear invalid input
            continue;
        }

        scanner.nextLine(); // consume leftover newline

        switch (choice) {
            case 1:
                System.out.print("Task name: ");
                String name = scanner.nextLine();

                System.out.print("Priority (1=High, 2=Medium, 3=Low): ");
                int priority;

                try {
                    priority = scanner.nextInt();
                } catch (InputMismatchException e) {
                    System.out.println("Priority must be a number.");
                    scanner.nextLine();
                    break;
                }

                manager.addTask(name, priority);
                break;

            case 2:
                manager.listTasks();

```

```

        break;

    case 3:
        System.out.print("Enter task index to mark done: ");
        int index;

        try {
            index = scanner.nextInt();
        } catch (InputMismatchException e) {
            System.out.println("Index must be a number.");
            scanner.nextLine();
            break;
        }

        manager.markTaskDone(index);
        break;

    case 4:
        System.out.println("Goodbye.");
        return;

    default:
        System.out.println("Invalid choice.");
    }
}
}
}
}
}

```

What this fixes

- Prevents crashes when the user types letters instead of numbers
- Clears invalid input so the loop continues normally
- Makes the menu fully robust for real users

If you want, I can also help you add features like

- input validation for priority values,
- looping until a valid number is entered, or
- cleaner menu handling.

Add a `deleteTask` method to `TaskManager` that removes a task by its index, and update `Main.java` to include a menu option for deleting a task. Do not change anything else. Here's the **minimal set of changes** you asked for:

- ✓ Add a **deleteTask** method to **TaskManager**
- ✓ Add a **menu option** in **Main.java** to call it
- ✓ **Do not change anything else**

No restructuring, no extra features — just the required additions.

✓ Updated **TaskManager.java** (only the new method added)

```
java
public void deleteTask(int index) {
    if (index >= 0 && index < tasks.size()) {
        tasks.remove(index);
        System.out.println("Task deleted.");
    } else {
        System.out.println("Invalid task index.");
    }
}
```

✓ Updated **Main.java** (only the new menu option + input handling)

Below is the **only required modification**:

- Add menu option **5. Delete Task**
- Add a **case 5** that asks for an index and calls **deleteTask**

```
java
case 5:
    System.out.print("Enter task index to delete: ");
    int deleteIndex;

    try {
        deleteIndex = scanner.nextInt();
    } catch (InputMismatchException e) {
        System.out.println("Index must be a number.");
        scanner.nextLine();
        break;
    }
```

```
manager.deleteTask(deleteIndex);  
break;
```

And add this line to your menu:

Code

5. Delete Task

If you want, I can help you next with

- **validating task indices**
- **preventing duplicate task names**
- **adding confirmation before deleting**